

Nama : Rika bagus Wijaya
Nim : G.231.24.0001
Matkul : Analisis kompleksitas algoritma

Tugas praktikum 7

1. Buatlah coding untuk fungsi yang menerapkan algoritma PCA
2. Tidak boleh menggunakan library dari python
3. Tidak boleh plagiasi teman
4. Upload melalui share github dengan nama FungsiPCA_NIM.ipynb

Pseudo code

hitung mean setiap kolom

lakukan centering data
 $\text{data_baru} = \text{data} - \text{mean}$

hitung covariance matrix

tentukan eigen vector
menggunakan power iteration

proyeksikan data
 $\text{hasil} = \text{data_baru} \times \text{eigen_vector}$

tampilkan hasil

Code pca

```
import math
```

```
def transpose(matrix):  
    hasil = []  
  
    for i in range(len(matrix[0])):  
        baris = []  
  
        for j in range(len(matrix)):  
            baris.append(matrix[j][i])  
  
        hasil.append(baris)
```

```
return hasil
```

```
def multiply_matrix(a, b):
```

```
    hasil = []
```

```
    for i in range(len(a)):
```

```
        baris = []
```

```
        for j in range(len(b[0])):
```

```
            total = 0
```

```
            for k in range(len(b)):
```

```
                total += a[i][k] * b[k][j]
```

```
        baris.append(total)
```

```
    hasil.append(baris)
```

```
return hasil
```

```
def mean_column(data):
```

```
    mean = []
```

```
    for j in range(len(data[0])):
```

```
        total = 0
```

```
        for i in range(len(data)):
```

```
            total += data[i][j]
```

```
    mean.append(total / len(data))
```

```
return mean
```

```
def center_data(data, mean):
```

```
    hasil = []
```

```
    for row in data:
```

```
        baris_baru = []
```

```
        for i in range(len(row)):
```

```
            baris_baru.append(row[i] - mean[i])
```

```

        hasil.append(baris_baru)

    return hasil

def covariance_matrix(data):
    transpose_data = transpose(data)

    cov_matrix = []

    for i in range(len(transpose_data)):
        row = []

        for j in range(len(transpose_data)):
            total = 0

            for k in range(len(data)):
                total += transpose_data[i][k] * transpose_data[j][k]

            row.append(total / (len(data) - 1))

        cov_matrix.append(row)

    return cov_matrix

def normalize(vector):
    total = 0

    for x in vector:
        total += x * x

    panjang = math.sqrt(total)

    hasil = []

    for x in vector:
        hasil.append(x / panjang)

    return hasil

def matrix_vector(matrix, vector):

```

```
hasil = []

for i in range(len(matrix)):
    total = 0

    for j in range(len(vector)):
        total += matrix[i][j] * vector[j]

    hasil.append(total)

return hasil
```

```
def power_iteration(matrix, iterasi=100):
    vector = [1 for _ in range(len(matrix))]

    for _ in range(iterasi):
        hasil = matrix_vector(matrix, vector)
        vector = normalize(hasil)

    return vector
```

```
def pca(data):
    mean = mean_column(data)

    centered = center_data(data, mean)

    cov_matrix = covariance_matrix(centered)

    eigen_vector = power_iteration(cov_matrix)

    eigen_matrix = [[x] for x in eigen_vector]

    hasil = multiply_matrix(centered, eigen_matrix)

    return hasil
```

```
data = [
    [2.5, 2.4],
    [0.5, 0.7],
    [2.2, 2.9],
    [1.9, 2.2],
```

```
[3.1, 3.0],  
[2.3, 2.7],  
[2.0, 1.6],  
[1.0, 1.1],  
[1.5, 1.6],  
[1.1, 0.9]  
]  
  
hasil_pca = pca(data)  
  
for row in hasil_pca:  
    print(row)
```

Program tersebut merupakan contoh penerapan algoritma PCA (Principal Component Analysis) yang dibuat secara manual tanpa memakai bantuan library khusus seperti NumPy maupun scikit-learn. Prosesnya dimulai dengan mencari nilai rata-rata dari setiap kolom data, lalu seluruh data disesuaikan terhadap rata-ratanya agar penyebaran datanya lebih mudah dianalisis. Setelah itu sistem membentuk covariance matrix untuk melihat seberapa kuat hubungan antar variabel pada data. Langkah selanjutnya yaitu mencari eigen vector menggunakan metode power iteration, karena bagian ini berfungsi menentukan arah utama penyebaran data yang paling dominan. Setelah principal component ditemukan, data kemudian diproyeksikan ke arah tersebut sehingga jumlah dimensinya bisa dikurangi namun informasi penting di dalam data tetap masih dapat dipertahankan.